

numOfLimitedObjects [SimTalk]

Counts all of the built-in objects which are contained in the entire model.

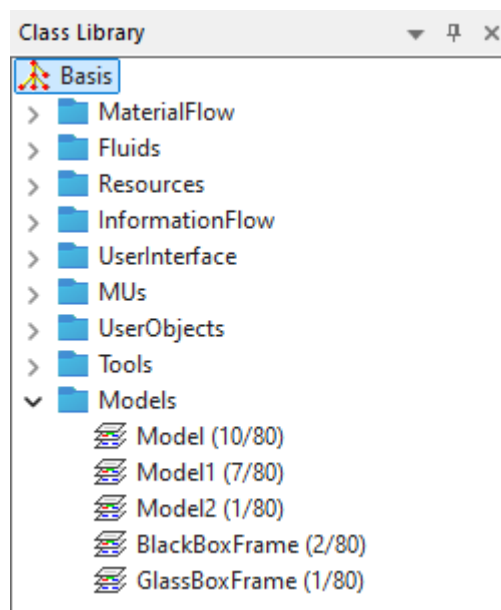
Remarks

numOfLimitedObject does not count *Lists* and *Tables*, *Methods*, *Variables*, *Comments*, *MUs*, *Connectors*, *Interfaces*, *EventControllers*, the *Toolbar*, *Folders*, and *Class* objects.

You might want to know this number if you are working with a *Plant Simulation Standard License* which is limited to 4000 objects.

For licenses, which limit the number of objects you can use, the *root object* **Basis** in the *Class Library* shows how many objects are used and how many are allowed for the license you are using at the moment.

For the **Student license** it might look like this. Our simulation model contains our *Frame* named **Model** and 10 objects which we inserted into it. 80 is the maximum number of objects which are allowed in the **Student license**.



Type

Function

Syntax

```
numOfLimitedObjects → integer
```

Return Value

The return value has the data type *integer*.

Example

```
print numOfLimitedObjects // returns 10 for the model in the screenshot
```

See also

[Console](#)

[Console Filter](#)

[Start Plant Simulation with Different Kinds of Licenses](#)

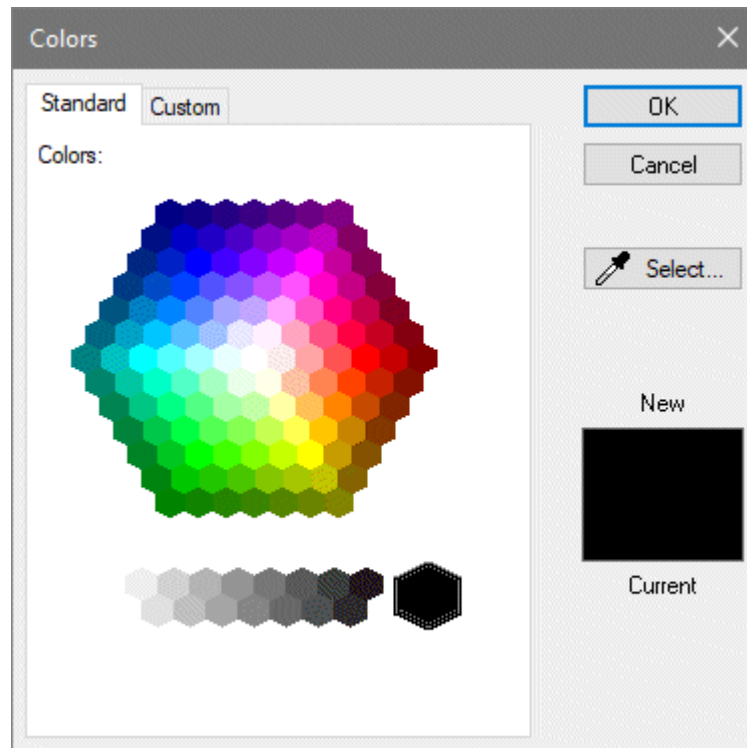
[NumberOfLimitedObjects \[SimTalk\]](#) of the *Frame*

openColorSelectBox [SimTalk]

Opens the dialog **Colors** provided by Windows.

Remarks

The dialog looks like this:

**Type****Function****Syntax**

```
openColorSelectBox(Color:integer) → integer
```

Parameter

The parameter *Color* of data type *integer* designates the active color value. Typically this will be a color which you define with the function *makeRGBValue*.

Return Value

The return value has the data type *integer*.

Is the RGB color value of the color selected by the user.

Example

```
colorVar := openColorSelectBox(colorVar)
openColorSelectBox(makeRGBValue(255,150,0)) // current color is orange
```

See also

[makeRGBValue \[SimTalk\]](#)

openConsole [SimTalk]

Opens the *Console window*.

Type

Function

Syntax

```
openConsole → boolean
```

Return Value

The return value has the data type *boolean*.

true if the *Console* was opened, false if it could not be opened.

Example

```
openConsole
```

See also

[Console](#)

[clearConsole \[SimTalk\]](#)

[closeConsole \[SimTalk\]](#)

[setConsoleFilter \[SimTalk\]](#)

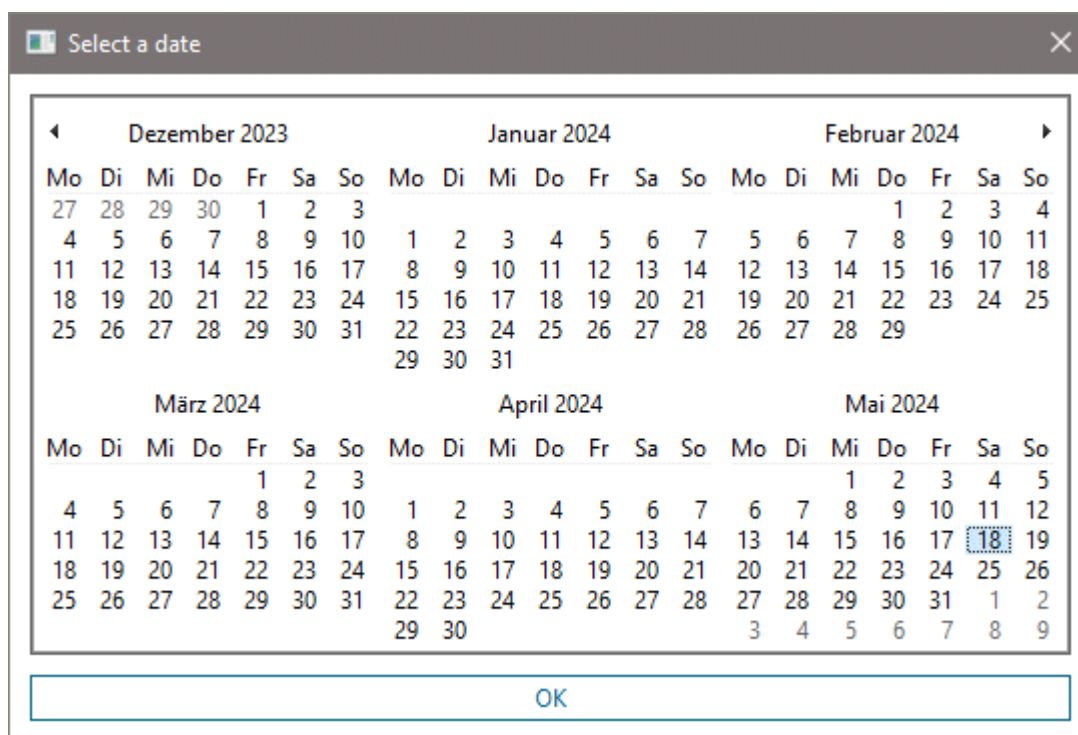
[Specifying Start Options](#)

openDateSelectBox [SimTalk]

Opens a dialog in which you can select a date.

Remarks

To set the date you select, click **OK**. If you close the dialog with the **Close** button in the title bar, *Plant Simulation* returns 1900/01/01, the first of January, 1900.



Type

Function

Syntax

```
openDateSelectBox(TitleText:string[, InitialDate:date]) → date
```

Parameters

You can specify the following parameters:

- The parameter *TitleText* of data type *string* designates the caption that the dialog shows in its title bar.
- The optional parameter *InitialDate* of data type *date* sets the date you select in the dialog.

Return Value

The return value has the data type *date*.

Is the date that the user picked.

Examples

```
openDateSelectBox("Select a date")  
print openDateSelectBox("Select a date") // returns 2024/05/18
```

```
openDateSelectBox("Select a date", str_to_date("2024-05-18"))  
print openDateSelectBox("Select a date") // returns 2024/05/18
```

openHTMLBrowser [SimTalk]

Starts the *HTML Browser* and opens the address designated by the parameter of data type *string*.

Remarks

To make the HTML Browser open a connection to the local calling *Plant Simulation* process, type an L or a forward slash / as the first character of the address.

URLs that begin with L/ only work when the web server has been started.

The safety setting **File > Model Settings > General > Prohibit Access to the Computer** affects the function *openHTMLBrowser* as follows:

- It permits only URLs with the protocols http, https, file, UNC paths and normal paths into the file system.
- It does not permit references to executable files for paths into the file system.
For *Plant Simulation* executable files are files whose file extension is listed in the environment variable PATH_EXT.
- For paths to files it only permits those paths that point to the model folder or its sub-folders.

Type

Function

Syntax

```
openHTMLBrowser(wwwAddress:string) → string
```

Parameter

The parameter *wwwAddress* of data type *string* designates the address to be opened.

Return Value

The return value has the data type *string*.

Examples

```
openHTMLBrowser("https://www.siemens.com/global/en.html")
```

```
openHTMLBrowser("https://www.siemens.com/global/de.html")
```


See also

[Prohibit Access to the Computer \[model settings\]](#)

openHTMLWindow [SimTalk]

Opens a browser window without the properties of a normal HTML browser window.

Remarks

The safety setting **File > Model Settings > General > Prohibit Access to the Computer** affects the function *openHTMLWindow* as follows:

- It permits only URLs with the protocols http, https, file, UNC paths, and normal paths into the file system.
- It does not permit references to executable files for paths into the file system.

For *Plant Simulation* executable files are files whose file extension is listed in the environment variable `PATH_EXT`.

- For paths to files it only permits those paths that point to the model folder or its sub-folders.

Microsoft Edge WebView2 can display documents in pdf (Portable Document Format) format. Provided *Microsoft Edge WebView2 Runtime* is installed on your computer, *openHTMLWindow* will show pdf documents directly in *Plant Simulation*.

Type

Function

Syntax

```
openHTMLWindow(Address:string, WindowTitle:string, X:integer, Y:integer,  
Width:integer, Height:integer) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Address* of data type *string* designates the address of the browser window.
- The parameter *WindowTitle* of data type *string* designates the title of the window.
- The parameter *X* of data type *integer* designates the x-coordinate at which the window opens on screen.
- The parameter *Y* of data type *integer* designates the y-coordinate at which the window opens on screen.
- The parameter *Width* of data type *integer* designates the width of the window.
- The parameter *Height* of data type *integer* designates the height of the window.

Return Value

The return value has the data type *boolean*.

Examples

```
openHTMLWindow("C:\Temp\mysite.htm", "My Page", 200,400,200,300)
```

```
openHTMLWindow("C:\Temp\document.pdf", "My pdf document", 200,400,200,300)
```

See also

[Prohibit Access to the Computer \[model settings\]](#)

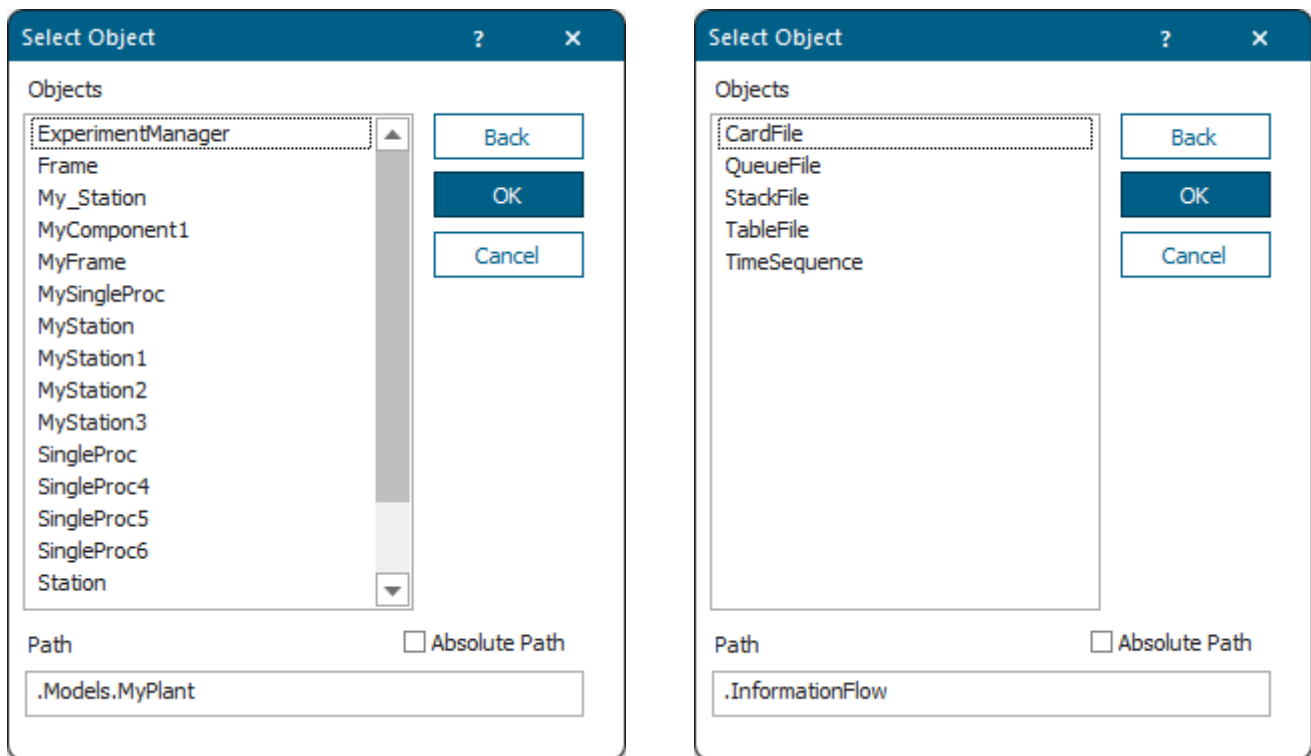
[closeHTMLWindow \[SimTalk\]](#)

openObjectSelectBox [SimTalk]

Opens the dialog **Select Object**.

Remarks

The dialog **Select Object** looks like this:



Type

Function

Syntax

```
openObjectSelectBox(Filter:string, FrameOrFolder:object) → string
```

Parameters

You can specify the following parameters:

- The parameter *Filter* of data type *string* designates the filter, i.e., the objects which the list box below **Objects** shows.
 - Specify an empty string "" to show all objects.

- Specify the *InternalClassType* to only show objects of that type.
- Specify `":Table"` to only show lists and tables.
- Specify `":MatCarrier"` to only show material flow objects.
- Specify `":Importer"` to only show material flow objects that have *importers*.

You can also enter several of these object types separated by colons.

Note

The internal identifiers for the classes are case-sensitive.

- The parameter *FrameOrFolder* of data type *object* designates the *Frame* or the *folder* for which the dialog **Select Object** will be initially opened.

Return Value

The return value has the data type *string*.

Examples

```
var path := openObjectSelectBox("Station", current)
// only shows the objects of type Station in the 'Select Object' dialog
// which are inserted into the simulation model
```

```
var path2 := openObjectSelectBox(":Table", .InformationFlow)
// shows all lists and tables in the 'Select Object' dialog
```

See also

[Select Object \[for controls\]](#)

[InternalClassType \[SimTalk\]](#)

processTime [SimTalk]

Returns the CPU time, which *Plant Simulation* used, in seconds.

Type

Function

Syntax

```
processTime -> real
```

Return Value

The return value has the data type *real*.

The return value has a resolution of several milliseconds.

Example

```
print processTime
```

See also

[getHighResolutionClock \[SimTalk\]](#)

[sysDate \[SimTalk\]](#)

profiler [SimTalk]

Activates or deactivates the *Profiler*.

Remarks

While it is active, it continuously collects information pertaining to the frequency of calls and the runtime of methods.

Type

Function

Syntax

```
profiler(Activate:boolean)
```

Parameter

The parameter *Activate* of data type *boolean* activates (`true`) or deactivates (`false`) the *Profiler*.

Example

```
profiler(true) // activate Profiler
```

See also

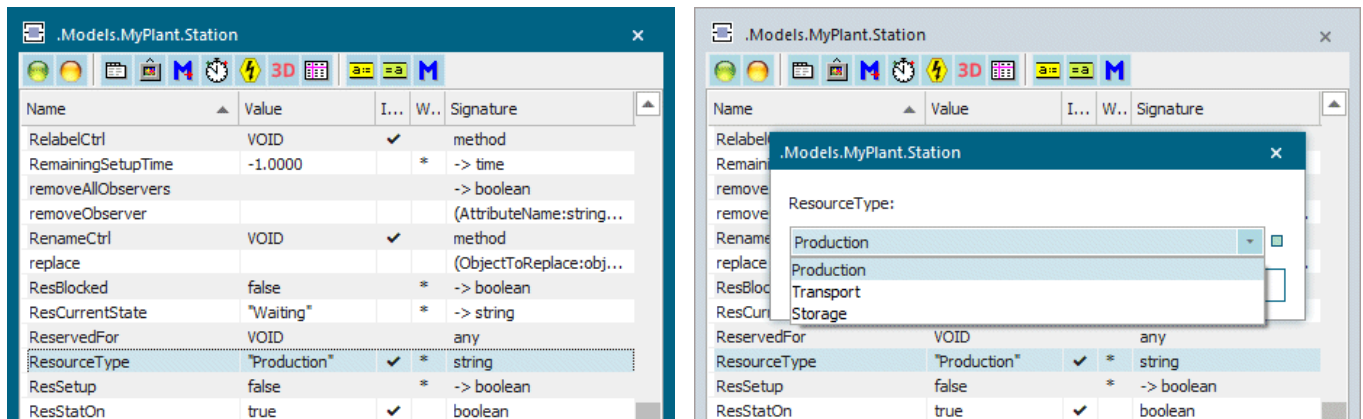
Activate Profiler	getProfileCallCycles [SimTalk]
Show Profiler Data [button]	resetProfile [SimTalk]
Reset Profiler	saveProfile [SimTalk]

putValuesIntoTable [SimTalk] - function

Writes the values of an attribute of data type *string*, which can only take certain values predefined in *Plant Simulation*, into a table.

Remarks

Examples for such attributes are the values of the drop-down lists in the dialogs of the objects. You can also access these settings in the window **Show Attributes and Methods** by double-clicking the respective attribute and selecting a value in the drop-down list.



The attribute *ResourceType* of the material flow objects can, for example, take the values **Production/Produktion**, **Transport/Transport**, or **Storage/Lagerung**, depending on the **Model Language** and the setting you selected for **Display Down-list Boxes** in the **SimTalk Language of the Model**.

The function returns the values according to the **Model Language** you selected and which you can also query with the function *language*.

Type

Function

Syntax

```
putValuesIntoTable(TargetTable:table)
```

Parameter

The parameter *TargetTable* of data type *table* designates the target table.

Example

```
var lst: list[string]
lst.create
MyStation.ResourceType.putValuesIntoTable(lst)
MyDialog.setList("My drop-down list box", lst)
```

The result looks like this after the values have been filled into the drop-down list:



Note

You can query if the value, which you enter, is selected without having to query the selected model language with the function *isSet*.

See also

[Show Attributes and Methods \[described\]](#)

[ResourceType \[SimTalk\]](#)

[Display Down-list Boxes in the SimTalk Language of the Model](#)

[Model Language \[model settings\]](#)

[language \[SimTalk\]](#)

[isSet \[SimTalk\]](#)

rad2deg [SimTalk]

Converts the specified angle in radians into the respective angle in degrees.

Title

Function

Syntax

```
rad2deg(AngleInRadians:real) -> real
```

Parameter

The parameter *AngleInRadians* of data type *real* designates the angle in radians which you would like to convert into the angle in degrees.

Return Value

The return value has the data type *real*.

Example

```
print rad2deg(0.785398163397448) // returns 45
```

See also

[deg2rad \[SimTalk\]](#)

readStringFromFile [SimTalk]

Reads the entire contents of the file and returns it as a value of data type *string*.

Remarks

If the file starts with a BOM (Byte Order Mark), it will be evaluated and not returned as part of the string.

Type

Function

Syntax

```
readStringFromFile(FileName:string) -> string
```

Parameter

The parameter *FileName* of data type *string* designates the file whose contents you would like to read.

Return Value

The return value has the data type *string*.

Example

```
var s: string := readStringFromFile(ApplicationHome +  
"Templates\English\Interaction\Interaction_connect.txt")  
print s
```

See also

[writeStringToFile \[SimTalk\]](#)

[bytes_to_str \[SimTalk\]](#)

resetProfile [SimTalk]

Deletes all data collected by the *Profiler* up until the time at which you reset the model.

Remarks

Call this function before activating the *Profiler* to only record valid data for the active simulation run.

Type

Function

Syntax

```
resetProfile
```

Example

```
resetProfile
```

See also

Activate Profiler	profiler [SimTalk]
Show Profiler Data [button]	saveProfile [SimTalk]
Reset Profiler	

resetRandomNumberStream [SimTalk]

Resets a random number stream for the *distribution function*, such as *z_uniform*, *z_normal*, etc.

Remarks

Each of the material flow objects automatically uses a random number stream of its own. You can set that random number stream with the attribute *RandomSeed* . You cannot access the random number streams of the objects with *resetRandomNumberStream* though. The random number stream, that the function passes, refers to the *distribution function*, such as *z_uniform*, *z_normal*, etc.

Type

Function

Syntax

```
resetRandomNumberStream(Stream:integer)
```

Parameter

The parameter *Stream* of data type *integer* designates the random number stream.

Example

```
resetRandomNumberStream(5)
```

See also

[Distribution Functions, z_ functions](#)

[RandomSeed \[SimTalk\] - material flow objects](#)

[Random Number Seed Values](#)

[setRandomSeedCounter \[SimTalk\]](#)

[setSeedTable \[SimTalk\]](#)

[getSeedTable \[SimTalk\]](#)

saveFolderModel [SimTalk]

Saves the simulation model in a folder/file structure which facilitates storing your model in a version control system such as *Git*.

Type

Function

Syntax

```
saveFolderModel(FileName:string[, CompactFormat:boolean:=false,  
UseGit:boolean:=false])
```

Parameters

You can specify the following parameters:

- The parameter *FileName* of data type *string* sets the path to and the name of the model.
- The optional parameter *CompactFormat* of data type *boolean* sets if *Plant Simulation* saves the objects inside of *Frames*, which have an origin, to an individual **.yaml** file (*true*) or not (*false*).

For the default setting *false* all the objects are saved into the **\$.yaml** file of the *Frame*. This reduces the number of files stored in the file system. This can be important for very large models to get a reasonable performance.

Default Value of the Parameter

The default value is *false*.

- The optional parameter *UseGit* of data type *boolean* sets if *Plant Simulation* creates a **Git repository** when the folder model is saved for the first time (*true*) or not (*false*).

The initial version will be committed automatically. This option is only available if *Git* is installed on your computer. In addition a **.gitignore** file is created in the top level folder. The content of this file ignores the **.UserSettings.yaml** so that it is not put under version control.

Default Value of the Parameter

The default value is *false*.

Git Commit Support

When you save a model as a folder model, you can set with the registry key **FolderModelCommitCommand** of type **String Value** which command will be called when saving the model.

Create the registry key either under **HKEY_CURRENT_USER\Software\Siemens\Tecnomatix Plant Simulation 2606** or under **HKEY_LOCAL_MACHINE\Software\Siemens\Tecnomatix Plant Simulation 2606**.

If the registry key exists, *TortoiseGit* will be started. If the registry key does not exist, *TortoiseGit* will not be started.

To never open the Commit dialog when saving the simulation model, enter an empty String "".

If you are using another source control system than *Git*, enter the path to that source control system as the **Value data**.

Example

```
saveFolderModel("D:\MyModels\MyDeportioner.psfm", true, true)
```

See also

[Save As > Folder Model](#)

[loadModel \[SimTalk\] - function](#)

saveModel [SimTalk]

Saves the current model with the file name you specify.

Remarks

If the file name does not specify a directory, *Plant Simulation* saves the model into the current directory which you can set with the function *setCurrentDirectory*.

If you did not set the current directory, it will be the directory into which you last saved a *Plant Simulation* model or a *Plant Simulation* item (icon, data table, etc.).

The method *modelFile* returns the current name of the model and the current folder of the model file.

Note

If you call the function *saveModel* within a formula or within an *executeSilent* command, *Plant Simulation* does not save any call chains into the model. This means that it does not save the currently called *Method* or the already scheduled *Methods* nor suspended *Methods*.

If you deactivate the safety setting **File > Model Settings > General > Prohibit Access to the Computer**, the function *saveModel* can copy data from another folder and write it to the model folder or its sub-folders. The function cannot copy files from the model folder or its sub-folders.

Type

Function

Syntax

```
saveModel(ModelName:string[, StopMethodExecution:boolean[,
UseNewName:boolean[, SaveCallChains:boolean]]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *ModelName* of data type *string* designates the name of the model file. This can be a fully qualified path.
- The optional parameter *StopMethodExecution* of data type *boolean* sets if *Plant Simulation* stops executing methods (*true*) or if method execution will be continued (*false*) after loading the model. If method execution is stopped, *Plant Simulation* opens the *Method Debugger* and highlights the line after the *saveModel* instruction. The default setting is *false*.

The simulation is stopped in both cases after loading the model. The parameter *StopMethodExecution* of data type *boolean* only affects method execution.

- The optional parameter *UseNewName* of data type *boolean* sets if the current model is to use the file name designated by the parameter of data type *string* (*true*), or if the model is to keep its name (*false*).

Default Value of the Parameter

The default value is *true*.

- The optional parameter *SaveCallChains* of data type *boolean* sets if all call chains in the model file are going to be saved (*true*) or not (*false*).

When you specify *false* *Plant Simulation* does not save the *Methods* being executed at the moment and suspended *Methods*.

Default Value of the Parameter

The default value is *true*.

Return Value

The return value has the data type *boolean*.

Example

```
print saveModel("C:\Temp\MyModel1.spp", false)

if not EventController.isRunning // was the model just loaded?
  if messageBox("Continue the simulation?",48,2) = 16 // Yes?
    EventController.start
  end
end
end
```

See also

[Prohibit Access to the Computer \[model settings\]](#)

[Save Model File](#)

[setCurrentDirectory \[SimTalk\]](#)

[closeModel \[SimTalk\] - global function](#)

[loadModel \[SimTalk\] - function](#)

[modelFile \[SimTalk\]](#)

saveProfile [SimTalk]

Saves all data collected by the *Profiler* to the designated file. Any existing contents of the file will be overwritten.

Remarks

You can print this file or load it into a *Method* object.

If you deactivate the safety setting **File > Model Settings > General > Prohibit Access to the Computer**, the function *saveProfile* can copy data from another folder and it to the model folder or its sub-folders. The function cannot copy files from the model folder or its sub-folders.

Type

Function

Syntax

```
saveProfile(FileName:string[, IncludeTop50CallCycles:boolean]) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *FileName* of data type *string* designates the path to the file. Any existing content of the file will be overwritten.
- The optional parameter *IncludeTop50CallCycles* of data type *boolean* sets if the **Top 50 Call Cycles** will be written to the file (*true*) or not (*false*).

Return Value

The return value has the data type *boolean*.

Example

```
saveProfile("C:\Users\Jeff\MyRun1.txt")
```

See also

Prohibit Access to the Computer [model settings]	Top 50 Call Cycles
Activate Profiler	profiler [SimTalk]

Show Profiler Data [button]	resetProfile [SimTalk]
Reset Profiler	

sendSMTPMail [SimTalk]

Sends an e-mail via the denoted mail server to the designated address.

Remarks

If you deactivate the safety setting **File > Model Settings > General > Prohibit Access to the Computer**, *Plant Simulation* does not execute the function *sendSMTPMail* and shows an error message.

Type

Function

Syntax

```
sendSMTPMail(Mail-Server:string, Receiver:string, Subject:string,  
MessageText:string)
```

Parameters

You can specify the following parameters:

- The parameter *Mail-Server* of data type *string* designates the mail server.
- The parameter *Receiver* of data type *string* designates the receiver of the mail.
- The parameter *Subject* of data type *string* designates the subject of the mail.
- The parameter *MessageText* of data type *string* designates the message text of the mail.

Example

```
sendSMTPMail("mail.company.com", "John.Smith@abc.org", "Test", "Hello John,  
the smoke test was successful.")
```

See also

[Prohibit Access to the Computer \[model settings\]](#)

setAntitheticRandomNumbers [SimTalk]

Sets if *Plant Simulation* uses **antithetic random numbers** (true) or **normal random numbers** (false).

Type

Function

Syntax

```
setAntitheticRandomNumbers(AntitheticRandomNumbers:boolean) → boolean
```

Parameter

The parameter *AntitheticRandomNumbers* of data type *boolean* sets if *Plant Simulation* uses **antithetic random numbers** (true) or **normal random numbers** (false).

Return Value

The return value has the data type *boolean*.

The return value shows the previous setting.

Example

```
setAntitheticRandomNumbers(true)
```

See also

[Antithetic Random Numbers](#)

setConsoleFilter [SimTalk]

Sets if the *Console* activates the designated *Console filters* (true) or not (false).

Type

Function

Syntax

```
setConsoleFilter(Filter:string, Activate:boolean) → boolean
```

Parameters

You can specify the following parameters:

- The parameter *Filter* of data type *string* designates the filter.
 - **Info**: Information about the model without user input.
 - **Message**: Information with user input from dialog windows and message boxes.
 - **Print**: Information from `print` commands in *Methods*.
 - **Debug**: Internal program messages for our software engineers.

You can also enter several filters, separated by the pipe symbol |.
- The parameter *Activate* of data type *boolean* sets if the filter will be activated (true) or not (false).

Return Value

The return value has the data type *boolean*.

Example

```
setConsoleFilter("Info|Message|Debug", false)
```

SimTalk

[clearConsole \[SimTalk\]](#)

[closeConsole \[SimTalk\]](#)

[openConsole \[SimTalk\]](#)

See also

[Console](#)

[Console Filter](#)

[Specifying Start Options](#), -logfile <file name>, for example -logfile MyLogFile.txt

[Specifying Start Options](#), -NoWarning

setEpsilon [SimTalk]

Sets the value for which *Plant Simulation* considers numerical values to be about equal.

Type

Function

Syntax

```
setEpsilon(Value:real) → real
```

Parameter

The parameter *Value* of data type *real* designates the value.

Note

Use the *about equal* comparison to prevent rounding errors when using *real* values.

Specify `print (1/3+1000-1000)*3` and see which number your computer returns.

Return Value

The return value has the data type *real*.

Example

```
setEpsilon(0.0001)
var a : real := 1.0003
var b : real := 1.00003
if a ~= 1.0 // this condition is false
  print "a is about equal to 1"
end
if b ~= 1.0 // this condition is true
  print "b is about equal to 1"
end
```

See also

[Tolerance For About Equal \(~=\) Comparison \[model settings\]](#)

[getEpsilon \[SimTalk\]](#)

setInfiniteLoopDetectionTimeout [SimTalk]

Sets the number of seconds which method execution might take before *Plant Simulation* opens the dialog telling you that you can stop method execution by pressing **Shift+Alt+Ctrl**.

Type

Function

Syntax

```
setInfiniteLoopDetectionTimeout(Timeout:integer) → integer
```

Parameter

The parameter *Timeout* of data type *integer* designates the number of seconds for the timeout. Specifying 0, or a negative value, will deactivate infinite loop detection entirely. Otherwise the allowed minimum value is 3 seconds.

Return Value

The return value has the data type *integer*.

Is the previous value.

Example

```
setInfiniteLoopDetectionTimeout(60)
```

setMUTraceRouteMethod [SimTalk]

Designates a *Method* or an *array_of_Methods*, which is called each and every time, when a *MU* is moved to another object.

Remarks

Methods which are registered with *setMUTraceRouteMethod* are removed from the *Registry* when you reset the *EventController*. Each model has to register its *Methods* anew when initializing the model. This prevents that *Methods* are executed for a model, which does not use *setMUTraceRouteMethod*, and which were registered by another model.

Type

Function

Syntax

```
setMUTraceRouteMethod(&Method:object) → object
setMUTraceRouteMethod([array_of_Methods:any]) → object
setMUTraceRouteMethod → object
```

Parameters for a Single Method Object

The method has four parameters.

- In the example below the parameter *fromObject* of data type *object* designates the object on which the *MU* was located before it was moved.
- The parameter *fromLength* of data type *length* designates the position of the *MU* on a *length-oriented object* at which the *MU* was located before it was moved.
- The parameter *fromLane* of data type *integer* designates the lane on which the *MU* was located on a *TwoLaneTrack*.
- The parameter *time* of data type *time* designates the point in time at which the *MU* is to be moved.

Return Value

The return value has the data type *object*.

Example

```
param fromObject: object, fromLength: length, fromLane: integer, t: time
var numSankeys: integer
var sankeylist: object
var toLength: length
sankeylist := rootfolder.internal.ActiveSankeyObjects
```

```

numSankeys := sankeylist.dim
for var i := 1 to numSankeys
    sankeylist.read(i).printTrace(@, fromObject, fromLength, fromLane, ?, t)
next

```

Parameters for An Array of Methods

The parameter *array_of_Methods* of data type *any* designates an *array* of *Method* objects.

You can also call the function *setMUTraceRouteMethod* without a parameter. If you specify an *array* of *Method* objects, *Plant Simulation* enters all *Methods* as **Callback Methods** and removes all previously existing **Callback Methods** from the list of **Callback Methods**.

If you call the function *setMUTraceRouteMethod* without parameter, it does not change the previously existing **Callback Methods** but returns an *array* of the previously existing **Callback Methods**.

Return Value

Is an *array* of data type *object* which contains all previously existing **Callback Methods**.

Example

```

// add 'MyMethod' to the Callback Methods
var callbacks: object[] := setMUTraceRouteMethod() // get the already
installed Callback Methods
callbacks.append(&MyMethod) // add MyMethod
setMUTraceRouteMethod(callbacks) // set the previously installed methods
plus MyMethod as Callback Methods

// remove 'MyMethod' from the Callback Methods
callbacks := setMUTraceRouteMethod() // get the installed Callback Methods
callbacks.deleteValue(&MyMethod) // remove MyMethod from the array
setMUTraceRouteMethod(callbacks) // set the previously installed
methods minus MyMethod as Callback Methods

// clear all Callback Methods
var empty: object[]
setMUTraceRouteMethod(empty) // setting an empty array will uninstall all
Callback Methods

```

setPythonDLLPath [SimTalk]

Specifies the path to the Python installation to be use if you are have multiple Python environments on your computer.

Remarks

Note

Plant Simulation supports Python versions 3.12, 3.13, and 3.14.

If you installed several Python versions on your computer, use *setPythonLLPath* to choose which version you want to use.

Type

Function

Syntax

```
setPythonDLLPath(dllpath:string)
```

Parameter

The parameter *dllPath* of data type *string* designates the path to the Python dll that you want to use.

Example

```
setPythonDLLPath("C:\Program Files\Python312\python312.dll")  
-- uses the Python version for all users, located in C:\Program  
Files\Python312, which is the default setting
```

```
setPythonDLLPath("C:\Users\<YourLoginName>\AppData\Local\Programs\Python\Pyt  
hon312\python312.dll")  
-- uses the Python version for the current user located in the folder above
```

See also

PythonModule

setRandomSeedCounter [SimTalk]

The function *setRandomSeedCounter* is only required if you want to influence the automatic allocation of the random number seed value, which normally is not required.

Remarks

When you insert an object into your model, *Plant Simulation* automatically assigns a random number seed value to this object. You can query the next assigned random seed value with the function *setRandomSeedCounter* (0). After the object is created, *Plant Simulation* increases this counter by 1, so that the next object gets a different random seed value.

This guarantees that the newly inserted object uses different random numbers, for example for **Processing Times**, for **Failures**, etc. To facilitate this, each model has a counter that *Plant Simulation* increases when automatically assigning the random number seed value. Calling the function *setRandomSeedCounter* with 0 only returns the current counter. If you call the function *setRandomSeedCounter* with a value other than 0, *Plant Simulation* sets the counter to this value.

Besides, the function *setRandomSeedCounter* returns the previous value of the counter. If you only want to get the current value of the counter, call the function with `CounterValue=0`.

If you set the counter to a value smaller than the previous value, it is not guaranteed that each newly inserted object has a unique random number seed value.

This function can be useful if you want to automatically create simulation models. You could, for example, use the *Teamcenter Interface* to detect the objects within your installation to then create these objects using a *Method*. When you then delete the created objects and get the up-to-date information from *Teamcenter* and create the model anew, the created object will get different random number seed values, even if the *Teamcenter* database did not change. To use the same random number seed values, call the function *setRandomSeedCounter* once before creating the objects to always initialize the counter with the same value.

Type

Function

Syntax

```
setRandomSeedCounter(CounterValue:integer) → integer
```

Parameter

The parameter *CounterValue* of data type *integer* designates the value of the counter.

Return Value

The return value has the data type *integer*.

Is the current random seed counter.

Example

```
setRandomSeedCounter(4)
print setRandomSeedCounter(0)
```

See also

[resetRandomNumberStream \[SimTalk\]](#)

[Random Numbers Variant](#)

[RandomSeed \[SimTalk\] - material flow objects](#)

setSeedTable [SimTalk]

Sets the **random number seed values** for creating the random number streams.

Remarks

Note

The seed values only apply to the *distribution functions*, such as *z_uniform*, *z_normal*, etc.

Each of the material flow objects automatically uses a random number stream of its own. You can set that random number stream with the attribute *RandomSeed*.

Note

Plant Simulation applies the new random numbers immediately and resets all random number streams with the new seed values.

Type

Function

Syntax

```
setSeedTable(SeedTable:table)
```

Parameter

The parameter *SeedTable* of data type *table* designates a table with one or two columns or a *list*.

- The first column contains the random number seed values (*integer*).
- The optional second column contains the comments to the random number seed values (*string*). The number of the row matches the number of the random number stream.

Examples

```
var t: table[integer]
t.create
t[1,6] := 1
setSeedTable(t)
```

```
var t: table[integer]
t.create
t[1,6] := 1
```

```
t[2,6] := "Explain usage"  
setSeedTable(t)
```

See also

[Random Number Seed Values](#)

[RandomSeed \[SimTalk\] - material flow objects](#)

[getSeedTable \[SimTalk\]](#)

[resetRandomNumberStream \[SimTalk\]](#)

[Distribution Functions, z_ functions](#)

showStatisticsReport [SimTalk]

Shows the **Statistics Report** of the objects which you type into the list.

Type

Function

Syntax

```
showStatisticsReport(List:list[, FileName:string])
```

Parameters

You can specify the following parameters:

- The parameter *List* of data type *list* designates the list into which you entered the names of the objects.
- The optional parameter *FileName* of data type *string* designates the name of the file into which *Plant Simulation* writes the statistics values.

Example

```
showStatisticsReport(myStatisticsObjects)  
showStatisticsReport(myStatisticsObjects,"My Statistics Report")
```

See also

[Statistics Report \[described\]](#)

strHash [SimTalk]

Calculates the hash value from the passed string.

Remarks

The value is located between 0 and 2147483647 (including both thresholds). The hash function is case-sensitive. If you do not want this, convert the string to lower case with the function *strToLower* beforehand.

You might, for example, use the hash value to find a string very fast. Comparing integer numbers is faster than comparing strings. The hash value generation is not a bijective function, i.e., it is not unambiguously reversible.

Once you have found the hash value, you should then compare the strings. If you want to map the hash value to a smaller value range, i.e., if you want to program a hash table, you should use the modulo value of the hash value.

Type

Function

Syntax

```
strHash(string) → integer
```

Return Value

The return value has the data type *integer*.

Example

```
var hashTable: any[7]
var emptyStringArray: string[]
for var i := 1 to hashTable.dim
    hashTable[i] := emptyStringArray
next
hashTable[strHash("Source") mod 7 + 1] := ["Source"]
hashTable[strHash("Drain") mod 7 + 1] := ["Drain"]
hashTable[strHash("AssemblyStation") mod 7 + 1] := ["AssemblyStation"]

var SourceFound: boolean := hashTable[strHash("Source") mod 7 + 1].find("Source") /= 0
var BufferFound: boolean := hashTable[strHash("Buffer") mod 7 + 1].find("Buffer") /= 0
```

See also

[strToLower \[SimTalk\]](#)

[computeSHA1Hash \[SimTalk\]](#)

[computeSHA3Hash \[SimTalk\]](#)

throwRuntimeError [SimTalk]

Returns an error message to the calling *Method* in library methods. This is, for example, the case if the library method was called with parameter values that are not allowed.

Remarks

You can also use the function outside of libraries. When the function *throwRuntimeError* is called, *Plant Simulation* opens the *Method Debugger* for the calling *Method* and shows the message that was passed to *throwRuntimeError*. In case the *Method* does not run as a subroutine, i.e., if no caller exists, *Plant Simulation* opens the *Method Debugger* at the position which contains the function *throwRuntimeError*. If the method is encrypted, *Plant Simulation* opens a message box instead that shows the message.

If the *Method* that executes *throwRuntimeError* is part of a library, *Plant Simulation* returns the call chain up to the point at which the first *Method* was found which does not belong to this library. For this *Method* *Plant Simulation* then opens the *Method Debugger*. If the *Method* that executes *throwRuntimeError* does not belong to a library, *Plant Simulation* opens the *Method Debugger* for the first direct caller of the *Method*.

If an error handling method (*ErrorHandler*) exists in the calling *Method* or in one of its callers, *Plant Simulation* does not open the *Method Debugger*, but instead calls the error handling method. That means that the search for an error handling method starts at the *Method* for which the *Method Debugger* would open. If this *Method* does not have an error handling method, *Plant Simulation* goes back in the call chain until it finds a local error handling method. If none of these *Methods* in the call chain has a local error handling method, it instead calls the global error handling method. If no global error handling method exists either, then *Plant Simulation* opens the *Method Debugger* or the message window as described above.

Type

Function

Syntax

```
throwRuntimeError(ErrorMessage:string)
```

Parameter

The parameter *ErrorMessage* of data type *string* sets the error message that *Plant Simulation* shows.

Example

```
param NewPosition : length
// the method drives the hook of the crane to a new position
if NewPosition < 0 or NewPosition > MyCrane.MyCraneArmLength
```

```
    throwRuntimeError("invalid crane position")  
end  
  
// set the new position ...
```

See also

[Error Handling](#)

[Calculate Values with a Formula](#)

[getCallStack \[SimTalk\]](#)

[setErrorHandler \[SimTalk\]](#)

updateGUI [SimTalk]

Updates the graphical user interface of *Plant Simulation*.

Remarks

If executing a *Method* takes a long time, it might seem that *Plant Simulation* stopped responding to your input. To prevent this from happening, you can every now and then call the function *updateGUI* within this *Method*.

updateGUI redraws the contents of the *Frames*.

Type

Function

Syntax

```
updateGUI -> integer  
updateGUI([forceUpdateNow:boolean:=false]) -> integer
```

Parameter

Specify the optional parameter *forceUpdateNow* and set it to `true` to enforce an immediate update of the user interface.

Do not specify the parameter or set it to `false` to only redraw the user interface one time after some time has passed since the last update.

Default Value of the Parameter

The default value is `false`.

Example

```
for var y := 1 to JTable.yDim  
  MyFrame._3D.importGraphics([0,y,0], JTable[1,y], false, true)  
  updateGUI  
next
```

userInterfaceLanguage [SimTalk]

Returns the language of the user interface of *Plant Simulation*.

Type

Function

Syntax

```
userInterfaceLanguage → integer
```

Return Value

The return value has the data type *integer*.

0 designates **German**, 1 **English**, 2 **Japanese**, 3 **Chinese**, and 8 designates **Hungarian**.

Example

```
if userInterfaceLanguage = 0      // German
  print "Guten Morgen"
elseif userInterfaceLanguage = 1  // English
  print "Good morning"
elseif userInterfaceLanguage = 2  // Japanese
  print "Ohayo gozaimasu"
elseif userInterfaceLanguage = 3  // Chinese
  print "Zao an"
elseif userInterfaceLanguage = 8  // Hungarian
  print "Jó reggelt"
end
```

See also

[language \[SimTalk\]](#)

[Model Language \[model settings\]](#)

[Specifying Start Options](#), /UILanguage:ENU, /UILanguage:DEU, /UILanguage:JPN, /UILanguage:CHS, or /UILanguage:HUN

writeStringToFile [SimTalk]

Writes the content of the specified *Text* to a file.

Type

Function

Syntax

```
writeStringToFile(Text:string, FileName:string[, Append:boolean,
Encoding:string:="UTF-8"])
```

Parameters

You can specify the following parameters:

- The parameter *Text* of data type *string* designates the text which you would like to write to the file.
- The parameter *FileName* of data type *string* designates the file or the directory path and the name of the file into which the file will be written. If the file exists, it will be overwritten.
- The optional parameter *append* of data type *boolean* sets if *Plant Simulation* deletes the existing content before writing (*false*) or if it does not delete it before writing and appends a line break (Carriage Return + Line Feed) to the written text (*true*).
- Do not specify the optional parameter *Encoding* of data type *string* or specify "UTF-8", to save the file in UTF-8-format with a BOM (Byte Order Mark).

Specify "Unicode" or "UTF-16" for *Encoding*, to save the file in UTF-16-format with a BOM.

Specify an empty string "" to save the file as UTF-8 without BOM.

Default Value of the Parameter

The default value is "UTF-8".

Examples

```
writeStringToFile("hello world", "C:\Users\Public\Public Documents\a.txt")
```

```
writeStringToFile(&MyMethod.Program, "C:\Temp\SourceCode.txt")
-- exports the source code of MyMethod to a text file
```

```
var data:json
data["Name"] = "John Doe"
data["Profession"] = "Simulation specialist"
```